

1

NOCK & THE ART OF
SOLID-STATE COMPUTING
An Enquiry into the Virtual

2

3

4

N. E. Davis
~lagrev-nocfep

5

6 Contents

7	1	The Roots of Computing in the Ancient	
8		World	1
9	1.1	Tabulation	3
10	1.2	Solid-state indices	3
11	2	Combinators for Fun and Profit	5
12	2.1	Combinators in Computing	6
13	2.1.1	The SKI calculus	6
14	2.1.2	The BCK ω calculus	7
15	2.2	Nock Opcodes as Combinator	8
16	2.2.1	I Identity = Opcode 0	8
17	2.2.2	K Constant = Opcode 1	8
18	2.2.3	S Substitution = Opcode 2	8
19	2.2.4	The Axiomatic Operators	8
20	2.3	Idioms & Design Patterns	8
21	2.3.1	C Reversal	9
22	2.4	Virtualization	9
23	3	Interpreter Runtime	11

Chapter 1

The Roots of Computing in the Ancient World

Abstract

Cuneiform and early computational concepts are explored in this chapter.

Machines are pure products of human intellect, constructed for man's own purposes. They are nowhere to be found in the world of nature (as products of nature); yet the workings of the laws of nature find their purest expression in machines, purer than in any of the products of nature itself. The laws of nature work directly in machines, with an immediacy not to be found in the products of nature.

42 (Keiji Nishitani, *Religion and Nothingness*,
43 1982, pp. 129–130)

44 There are many worthy raconteurs of the history
45 of computing, such as **Hofstadter1980 (Hofstadter1980)**
46 and **Penrose2004 (Penrose2004)**. What distinguishes
47 my approach to the history of computing from other
48 tellings? Computing has deep connexions to alchemy
49 and true language, which I hope to demonstrate with-
50 out assuming any Hermetic background on the reader's
51 part. Much like technology, computing is not merely
52 a tool but a stance towards the world. However, it
53 does not entail technologism's supercession; com-
54 puting is always both a lens and a kaleidoscope, through
55 which many layers of reality can be seen simultane-
56 ously. It is a way of seeing the world as a system
57 of pure statements and acting to make those state-
58 ments true. It is both ontology and epistemology.

59 First, therefore, I must establish that computa-
60 tion is the apex of a long human search for a lan-
61 guage of truth. The origin of symbol itself in the
62 σύμβολον, a token broken in half to infallibly mark
63 identity, is one font. Later "symbol" was the profes-
64 sion of faith, both an attempt to formulate truth and
65 a sign of shared claims. The primary source, how-
66 ever, is language itself, Hofstadter's "strange loop"
67 of self-reference. The first possible Nock formula is
68 a crash, the void. The second is self-reference, awak-
69 ening. Peirce would be proud of the progression. We
70 have tricked ourselves into thinking that computing
71 is a late development, a fruitful elaboration of the
72 end of mathematics. In fact, it is the philosopher's
73 stone, and it underlies every process in the world. It

74 has turned lead into gold.

75 The historical account which follows aims through-
76 out at the goal of understanding what Nock is and
77 why it matters as a protocol. The reader will under-
78 stand what mankind has been looking for, when we
79 have caught various pieces of the puzzle, how these
80 things came together in the mid-20th century, and
81 how they have been refined into a pure machine.

82 Outline: - Cuneiform, tabulation, calculation, and
83 algorithm - Solid-state indices: abaci, tokens, etc.
84 Mechanism as embodied equation. - Pure language
85 and true statements (Llull, Leibniz, Wilkins) - Gen-
86 eralization (Jacquard, Babbage) - The universal ma-
87 chine (Turing, Gödel, Zuse, von Neumann) - Pure
88 statements as Pages from the Book (Curry, Church)

89 1.1 Tabulation

90 1.2 Solid-state indices

91 The problem is that any calculation lives in some-
92 one's head or someone's claim. This is a summary,
93 but not the proof or solution.

94 Chapter 2

95 Combinators for Fun 96 and Profit

97 Abstract

98 The first section of this chapter will introduce the
99 basic SKI and BCKW combinator calculi and examine
100 how they can represent each other. The second sec-
101 tion will introduce Nock’s opcodes as combinators,
102 and show how they can be used to express various
103 combinator idioms. The third section will show how
104 to build practical affordances for common program-
105 ming tasks, such as conditionals, recursion, subpro-
106 cedure invocation, and data structures. The final sec-
107 tion will discuss the relationship between combina-
108 tors and virtualization, and how Nock can be used to
109 build a virtual machine even in a crash-only context.

110 2.1 Combinators in Computing

111 The pioneers of computing elaborated three comple-
112 mentary visions:

- 113 1. The Turing machine, a simple abstract machine
114 that can compute anything computable.
- 115 2. The lambda calculus, a pure functional language
116 based on substitution and abstraction.
- 117 3. Combinatory logic, a variable-free functional
118 language based on combinators.

119 Logicians early demonstrated the equivalence of
120 these approaches, but particular formulations are still
121 preferred for particular tasks. The Turing machine is
122 the most intuitive model of computation, and is of-
123 ten used to prove computability results. The lambda
124 calculus is the basis of many functional program-
125 ming languages, such as Lisp, Scheme, and Haskell.
126 Combinatory logic is less widely used in practice,
127 but has a long history in mathematical logic and type
128 theory. It is also the basis of Nock.

129 One of the most fundamental objectives of com-
130 binatory logic is to eliminate variables.

131 2.1.1 The SKI calculus

132 Computable functions can be expressed using mini-
133 mal sets of combinators. Many logicians today pre-
134 fer to work with Moses Schönfinkel and Haskell Curry's
135 SKI combinator calculus, which has three combina-
136 tors:

137 The identity combinator I is conventionally writ-
 138 ten $Ix = x$. This simply means that it returns the
 139 result of its evaluation unchanged.

140 K
 141 S

142 The SKI system is Turing complete (**blah, blah**),
 143 meaning that any computable function can be ex-
 144 pressed using only these three combinators. It is also
 145 equivalent to the lambda calculus, meaning that any
 146 lambda expression can be translated into an equi-
 147 valent SKI expression and vice versa. (cite church-
 148 turing thesis stuff)

149 2.1.2 The BCKW calculus

150 B
 151 C
 152 W
 153 U
 154 Y
 155 $Y = S (K (S I I)) (S (S (K S) K) (K (S I I)))$
 156 iota combinator

157 Combinator calculi are generally considered too
 158 minimalist for practical programming; they shine,
 159 however, in mathematical proofs. This means that
 160 we can use them to define a standard of behavior
 161 for an algorithm, but implement the logic in a com-
 162 pliant form using whatever computational tools we
 163 like. Thus it is better to think of Nock not as a bare
 164 programming language, but as a standard of compu-
 165 tation which can under many circumstances be used

166 as the computation itself.¹

167 2.2 Nock Opcodes as Combinator

168 2.2.1 I Identity = Opcode 0

169 Nock's binary tree addressing is fully general and
170 yields the identity combinator I as a special case.
171 (The cost, of course, is dragging in some machinery
172 for basic arithmetic, as we will see.)

173 Conceptually, we could write this in a hybrid form
174 between combinators and Nock thus:

175 $*[a \ I \ b] == [0 \ 1]$

178 2.2.2 K Constant = Opcode 1

179 2.2.3 S Substitution = Opcode 2

180 2.2.4 The Axiomatic Operators

181 Because of Nock's cellular nature as a binary tree,
182 we need the ability to access and manipulate nouns.

183 opcode 3, 4, 5, 10

184 2.3 Idioms & Design Patterns

185 Several of the combinators we examined above are
186 not present in Nock *simpliciter*, but we can build them

¹Consideration of when to do so is largely driven by performance bottlenecks.

187 out of the other pieces. There are also common id-
 188 ioms which are baked in as “compound opcodes” (such
 189 as conditional branching, opcode 6).

190 2.3.1 C Reversal

191 $C = S (S (K (S (K S) K)) S) (K K)$

192 In Nock, the reversal combinator C is almost free
 193 due to the binary tree nature of cells.

194
 195 $\star[a \ C \ b] == \star[7 \ [0 \ 3] \ [[0 \ 3] \ [0 \ 2]]]$

196 $\star[a \ C \ b] == \star[[0 \ 7] \ [0 \ 6]]$
 197

198 Recursion we get for free with opcode o.
 199 μ -recursive functions

200 2.4 Virtualization

201 Nock is a crash-only pure language, which means at
 202 the base level it can only succeed and yield a new
 203 noun: no side effects, no exceptions, no state. Like
 204 a castaway on a desert island, we must build every-
 205 thing we need from raw materials at hand.

206 Nock has one trick built in and a couple of design
 207 patterns to help us get there:

- 208 1. Opcode 11 lets us send a hint to the runtime,
 209 which can be used to trigger side effects. The
 210 Nock expression formally doesn’t know any-
 211 thing about these, but we can use them to print
 212 results, read input, and so forth.
- 213 2. The runtime can maintain state outside of Nock
 214 and supply it as a subject to subsequent Nock

215 expressions. We will do a lot more with this
216 later, but we want to highlight it as a key af-
217 fordance now.

218 3. The virtualization pattern means that we can
219 run Nock in Nock.

220 Opcode 11 can encode anything we want: the re-
221 sult of a hint is used by the runtime, and so anything
222 the runtime is willing to work with can be produced.

223 The magic formula at the beating heart of a prac-
224 tical Nock operating system is simply:

225

226 `★([state -.events] [9 2 10 [6 0 3] 0 2])`
227

228 That is, apply the Nock formula against a subject pair
229 consisting of the current state and the head of the list
230 of events to process.

231 Chapter 3

232 Interpreter Runtime

233 Abstract

234 Now that we have established a firm foundation
235 for idiomatic programming in Nock, we can build an
236 interpreter using a higher-level language to execute
237 Nock formulas. At this point, we will build the sim-
238 plest possible Nock interpreter, a tree-walking inter-
239 preter. This requires a cursor and some basic mem-
240 ory management but not much more. We will im-
241 plement some minimalist static hints to permit side
242 effects like output.

